

Supplementary material for the work “Electron-Ion Path Integral Monte Carlo”

Riccardo Fantoni*

Università di Trieste, Dipartimento di Fisica, strada Costiera 11, 34151 Grignano (Trieste), Italy

(Dated: May 13, 2026)

Supplementary material to the article “Electron-Ion Path Integral Monte Carlo”.

Keywords: Two-Component-Plasma; Electron-Ion Plasma; Fermi-Dirac Statistics; Restricted Path Integral Monte Carlo; Thermodynamics; Structure

I. INTRODUCTION

We here report the FORTRAN code listing of the code used in the simulations developed in the publication “Electron-Ion Path Integral Monte Carlo”. This is shown in Appendix A. The model studied in that publication was that of a 1:1 electron-proton plasma. A particular Two Component Plasma (TCP) where the two species are both fermions, one about 2000 times more heavy than the other. A particular neutral binary mixture with Coulomb pair interactions.

The Restricted Path Integral Monte Carlo (RPIMC) method used in our computer experiments is the one described in Ref. [1, 2]. But the permutation sampling necessary in calculating the determinant required by the study of the Fermi-Dirac statistics was carried out proposing swaps of a randomly chosen pair of particles through the Lévy construction of a pair of Brownian bridges (see Ref. [3] or Section V.G of Ref. [1] and references therein) connecting the two paths.

In the simulation we measure some thermodynamic properties like the total kinetic and potential energy and the superfluid fraction of each species. And some structural properties like the partial Radial Distribution Functions (RDF) for the two species.

The plane waves ¹ Path Integral Monte Carlo (PIMC) method used in our computer experiments is the one described in Ref. [1]. A single MC step is made of a swap of a randomly chosen pair of particles through the Lévy construction of two Brownian bridge between two randomly chosen timeslices on the two different paths (see Section V.G of Ref. [1] and references therein for the multislice move) and of a displacement of all the timeslices of a particle path chosen at random (the singleslice move). This allows the permutation sampling necessary in calculating the symmetric density matrix required by the study of the Bose-Einstein statistics. We then introduced a path restriction in order to overcome the “fermions sign problem” [5] based on the sign of the free fermions density matrix calculated in subroutine `ifdm`. This results in an approximate RPIMC calculation [2] which calculates the antisymmetric density matrix required by the study of

the Fermi-Dirac statistics. Of course we only apply the swap to like particles of the same species.

The extent of the singleslice displacement is chosen at the beginning of the run through the input variable `ALPHA` or it can be adjusted during the run every `IRATIO` MC steps so to have acceptance ratios closest to 1/2 (this functionality is commented out in the present listing).

We discretize the imaginary time between 0 and `BETA` into `FTN0` timeslices of length `LS=BETA/FTN0`. One usually wants to compare simulations at fixed `LS`. The multislice move requires the reconstruction of a randomly chosen number `MBM≤MBMM≤FNT0` of timeslices between a randomly chosen initial timeslice `CP` on a randomly chosen particle `IP` and the final timeslice on a randomly chosen particle `KP`. Here `MBMM` is the maximum number of timeslices in a Brownian bridge. The singleslice move requires the displacement of all the `FTN0` timeslices of the path of the particle `IP` chosen at random. So a single MC step requires a maximum of `2MBMM+FTN0` timeslice moves. In the code we wrote the FORTRAN instructions for the multislice move in lowercase and the ones for the singleslice move in uppercase. The simulation uses only the multislice move if the logical variable `IFBRIDGE` is `TRUE` and only the singleslice move if the logical variable `IFDISP` is `TRUE`. The simulation lasts for a total number of MC steps `NSTEP`. All these variables can be chosen in the `data-mix.in` simulation configuration input file read at startup.

The code works in any space dimension `DIM≤MDIM` with a mixture with a number of components `NMIX≤MMIX`. The number of particles of species `I` is `NPM(I)` and they have mass `FTM(I)`. One has to specify also the thermodynamic configuration giving the density `DENS` and the absolute temperature `TEMP`.

The pair potential between the particles `POTK` is encoded in SUBROUTINE `POT` at the end of the code listing. It includes the AQHS effective potential (`QHS`) and the canonical HS (`BUMP`). Other test cases included are the harmonic potential (`HARMONIC`), the Coulomb potential (`COULOMBHS` for the hard core Eq. (2.8) in the paper and `COULOMB` without the hard cores but just a cutoff at short distances), the penetrable square well potential (`PSW`), and the Lennard-Jones potential (`LJ`). The ideal gas is obtained choosing the string `FREE` in the `data-mix.in` simulation configuration input file. The potential can be further specified through the variables `RCUT`, `SIG`, `EPSR`, `EPSA`, and `EPS` in the simulation configuration input file.

* riccardo.fantoni@scuola.istruzione.it

¹ For a coherent states PIMC see Ref. [4].

For the sake of the present work we only use the Coulomb potential `COULOMBHS` where `SIG(I,J)` is given by Eq. (2.9) in the main text of the article and `EPSA(I,J)` is given by Eq. (2.10) in the main text of the article.

The simulation calculates both thermodynamic quantities and static structural properties. For the thermodynamic quantities we measure: the kinetic energy `KE`, the potential energy `V`, and the superfluid fraction of each species `SF(I)`. These quantities are stored in the output file `fort.9` every `IPRINT` MC steps. Whereas the path configurations output file `CNFILE` is stored in a `.xyz` format every `ISAVE` MC steps. The first `IEQUI` MC steps are used for the equilibration of the Markov chain. Quantities are only calculated after these equilibration steps. In order to determine efficiently the superfluid fraction at low temperature, which requires large `FTNO`, it is convenient to deactivate the displace move choosing `TRUE` for `IFBRIDGE`. For the structural properties we measure the partial radial distribution function $g_{ij}(r)$ which are written in the output files `rdf11.dat`, `rdf12.dat`, `rdf22.dat`. In the current implementation of the code, the $g_{ij}(r)$ are only calculated during the displacement move every `ICALCG` timeslices.

The logical variable `IFB` is set in the simulation configuration input file. If it is set to `TRUE` permutations of particles are sampled, otherwise only Boltzmann statistics for distinguishable particles is used.

If the variable `INIT` is set to 0 it starts from a path with the particles randomly distributed uniformly with no overlaps with all timeslices copied above the first one for each particle, if it is set to 1 it reads the paths configuration from the file `CNFILE` stored on disk, if it is set to 2 it does a complete restart from the file `CNFILE` and the frozen configuration `fort.20` both stored on disk every `ISAVE` MC steps.

The variable `SEED` is also set in the simulation configuration input file `data-mix.in`. It is the positive integer seed of the pseudo random number generator.

ACKNOWLEDGMENTS

I would like to thank prof. Saverio Moroni for his support in the development of the Brownian bridge which allows the particles permutation sampling.

Appendix A: The code

This is the code used for the PIMC computer experiment. We list here the main FORTRAN code `rpimc-mix.f` with its included `mc-mix.par` parameters file. And the input data file `data-mix.in` (for cases G and H of the Table in the main text of the article) that is read at the beginning of the run.

```
*****
*** rpimc-mix.f *****
*****
```

```
PROGRAM RPIMC
  IMPLICIT NONE
  *****
  ** MONTE CARLO SIMULATION PROGRAM FOR [PW] RESTRICTED PATH **
  ** INTEGRAL FOR A BINARY MIXTURE **
  **
  ** space dimensions: DIM=1,2,3 **
  ** number of particles: NP **
  ** number of timesteps: FTNO **
  ** units: hbar=k_B=1 **
  ** BETA=1/TEMP=FTNO*LS (LS imaginary time spacing) **
  **
  ** OBSERVABLES: **
  ** kinetic energy KE **
  ** potential energy V **
  ** forces virial W **
  ** superfluid fraction SF **
  ** radial distribution function in **
  ** 'rdf11.dat', 'rdf12.dat', 'rdf22.dat' **
  *****
  INCLUDE 'mc-mix.par' ! parameters
  *****
  INTEGER*4 SEED
  INTEGER*8 IDIM, STEP, I, J, K
  INTEGER*8 CI, CP, IP, KP, NIP, MKP, LL, OUT1, OUT2, PIP
  INTEGER*8 INIT, NSTEP, NS, IPRINT, ISAVE, IRATIO, IEQUI
  INTEGER*8 MBMM, MCM, PREV(MNP)
  INTEGER*8 RS

  REAL*8 DENS, TEMP, BETA, DENS LJ
  REAL*8 DRMAX, ALPHA, CDIM
  REAL*8 RANF, DUMMY, SR9, SR3
  REAL*8 VLRC, VLRCY, VLRC12, WLRC, WLRC6, WLRC12
  REAL*8 ACM, ACATMA, ACATMAS, ACATMAB
  REAL*8 V, VNEW, VOLD, VEND, DELTV, VN, VS
  REAL*8 W, WNEW, WOLD, WEND, DELTW, WN, WS
  REAL*8 KE, KENEW, KEOLD, DELTKE, KEEND
  REAL*8 AVV, ACV, ACVSQ, FLV
  REAL*8 AVV, ACW, ACWSQ, FLW
  REAL*8 AVKE, ACKE, ACKESQ, FLKE
  REAL*8 ACT, ACTNEW, ACTOLD, DELTACT, DELTACTB
  REAL*8 RXIOLD(MDIM), RXINEM(MDIM)
  REAL*8 RXP(MDIM,0:N), RXPP(MDIM,0:N)
  REAL*8 STD(MMIX), PS, KKK, VVV, WWW
  REAL*8 RATIO, RATIOS, RATIOB
  REAL*8 WIND(MMIX,MDIM), ACWINDSQ(MMIX,MDIM)
  REAL*8 SFI(MMIX), SF(MMIX), WSP(MMIX)
  REAL*8 SGN1, SGN2, RXOLD(MDIM,0:N,MNP)

  INTEGER*8 BIN, MAXBIN, MAXBIN2, ICALCG
  INTEGER*8 HIST11(MBIN), HIST12(MBIN), HIST22(MBIN)
  REAL*8 CONST, RLOWER, RUPPER, TSTEP
  REAL*8 NIDEAL11, NIDEAL12, NIDEAL22
  REAL*8 GR11, GR12, GR22, CO(MMIX)
  REAL*8 CONST11, CONST12, CONST21, CONST22

  CHARACTER CNFILE*30, POTK*10

  LOGICAL OVRLAP, IFB, IFDISP, IFBRIDGE

  C
  C PI = ACOS(-1.d0)
  C
  C OPEN (UNIT = 11, FILE = 'rdf11.dat', STATUS = 'UNKNOWN')
  C OPEN (UNIT = 12, FILE = 'rdf12.dat', STATUS = 'UNKNOWN')
  C OPEN (UNIT = 13, FILE = 'rdf22.dat', STATUS = 'UNKNOWN')
  C OPEN (UNIT = 14, FILE = 'rdf10.dat', STATUS = 'UNKNOWN')
  C
  C *****
  C ** READ INPUT DATA **
  C *****

  WRITE(*, '( ***** PROGRAM PIMC ***** )')
  WRITE(*, '( PLANE WAVES )')
  WRITE(*, '( PATH INTEGRAL MONTE CARLO PROGRAM )')
  OPEN (UNIT=10, FILE='data-mix.in', STATUS='UNKNOWN')
  READ (10,*) I
  READ (10,*) DIM, NMIX
  READ (10,*) I
  READ (10,*) SEED
  READ (10,*) I
  READ (10,*) IFB
  READ (10,*) I
  READ (10,*) (NPM(J), J=1, NMIX)
  READ (10,*) I
  READ (10,*) (A') POTK
  READ (10,*) I
  READ (10,*) FTNO
  READ (10,*) I
  READ (10,*) (FTM(J), J=1, NMIX)
  READ (10,*) I
  READ (10,*) NSTEP
  READ (10,*) I
  READ (10,*) IPRINT
  READ (10,*) I
  READ (10,*) ISAVE
  READ (10,*) I
  READ (10,*) IEQUI
  READ (10,*) I
  READ (10,*) IRATIO
  READ (10,*) I
  READ (10,*) ICALCG
  READ (10,*) I
  READ (10,*) (A') CNFILE
  READ (10,*) I
  READ (10,*) INIT
  READ (10,*) I
  READ (10,*) DENS
  READ (10,*) I
  READ (10,*) TEMP
  READ (10,*) I
  READ (10,*) ALPHA
  READ (10,*) I
  READ (10,*) MBMM
```

```

READ (10,*) I
READ (10,*) RCUT
READ (10,*) I
READ (10,*) ((SIG(I, J),I=1,NMIX),J=1,NMIX)
READ (10,*) I
READ (10,*) EPSR
READ (10,*) I
READ (10,*) ((EPSA(I, J),I=1,NMIX),J=1,NMIX)
READ (10,*) I
READ (10,*) EPS
READ (10,*) I
READ (10,*) IFDISP
READ (10,*) I
READ (10,*) IFBRIDGE
CLOSE (UNIT=10)

WRITE(*,(' IN DIMENSION (1,2,3,...)'I12/)) DIM
WRITE(*,(' ***** '))

GOTO 1111 ! comment if input from keyboard

WRITE(*,(' ***** PROGRAM PIMC ***** '))
WRITE(*,(' PLANE WAVES '))
WRITE(*,(' PATH INTEGRAL MONTE CARLO PROGRAM '))

WRITE(*,(' ENTER THE SPATIAL AND MIXTURE DIMENSIONS '))
READ (*,*) DIM, NMIX
WRITE(*,(' ENTER SEED FOR RANDOM SEQUENCE '))
READ (*,*) SEED
WRITE(*,(' IF BOSE (T/F) '))
READ (*,*) IFB
WRITE(*,(' ENTER THE NUMBER OF PARTICLES < MNP '))
READ (*,*) (NPM(I),I=1,NMIX)
WRITE(*,(' ENTER TYPE OF PAIR POTENTIAL (Many Body) '))
READ (*,*) (A) POTK
WRITE(*,(' ENTER THE NUMBER OF DISCRETIZATIONS < N '))
READ (*,*) FTNO
WRITE(*,(' ENTER THE BARE MASS '))
READ (*,*) (FTM(I),I=1,NMIX)
WRITE(*,(' ENTER NUMBER OF CYCLES '))
READ (*,*) NSTEP
WRITE(*,(' ENTER NUMBER OF STEPS BETWEEN OUTPUT LINES '))
READ (*,*) IPRINT
WRITE(*,(' ENTER NUMBER OF STEPS BETWEEN DATA SAVES '))
READ (*,*) ISAVE
WRITE(*,(' ENTER NUMBER OF STEPS FOR EQUILIBRATION '))
READ (*,*) IEQUI
WRITE(*,(' ENTER INTERVAL FOR UPDATE OF MAX. DISPL. '))
READ (*,*) IRATIO
WRITE(*,(' ENTER INTERVAL FOR RDF CALCULATION '))
READ (*,*) ICALOG
WRITE(*,(' ENTER THE CONFIGURATION FILE NAME '))
READ (*,*) (A) CNFILE
WRITE(*,(' ENTER 0 IF INITIALIZATION NEEDED '))
READ (*,*) INIT
WRITE(*,(' ENTER THE DENSITY '))
READ (*,*) DENS
WRITE(*,(' ENTER THE TEMPERATURE '))
READ (*,*) TEMP
WRITE(*,(' ENTER MAX. DISPLACEMENT DRMAX/SIGMA '))
READ (*,*) ALPHA
WRITE(*,(' ENTER MAXIMUM NUMBER OF BRIDGE TIMESLICES '))
READ (*,*) MBMM
WRITE(*,(' ENTER THE POTENTIAL CUTOFF DISTANCE ( ~ 2.5) '))
READ (*,*) RCUT
WRITE(*,(' ENTER SIG '))
READ (*,*) ((SIG(I, J),I=1,NMIX),J=1,NMIX)
WRITE(*,(' ENTER EPSR '))
READ (*,*) EPSR
WRITE(*,(' ENTER EPSA '))
READ (*,*) ((EPSA(I, J),I=1,NMIX),J=1,NMIX)
WRITE(*,(' ENTER EPS '))
READ (*,*) EPS
WRITE(*,(' IF DISPLACEMENT MOVE (T/F) '))
READ (*,*) IFDISP
WRITE(*,(' IF BRIDGE MOVE (T/F) '))
READ (*,*) IFBRIDGE
WRITE(*,(' IN DIMENSION (1,2,3,...)'I12/)) DIM
WRITE(*,(' ***** '))

1111 CONTINUE

IF (MBMM .GT. FTNO) THEN
  WRITE(*,*) "number of timeslices in bridge > ",FTNO
  STOP
ENDIF

NP = 0
DO I=1, NMIX
  NP = NP + NPM(I)
ENDDO
DO I=1, NMIX
  CO(I)=DBLE(NPM(I))/DBLE(NP)
ENDDO

C ** INITIALIZE RANDOM NUMBER GENERATOR **

CALL SRAND ( SEED )
IF (SEED.EQ.1) THEN
  CALL SRAND ( 0 )
ENDIF

C ** INVERSE TEMPERATURE BETA **

BETA = (1.d0/TEMP)

C ** IMAGINARY TIMESTEP **

LS = BETA/FTNO ! time step
DO I=1,NMIX
  STD(I) = (LS/FTM(I))*0.5 ! standatd deviation of free-particle rho

```

```

ENDDO

C ** WRITE INPUT DATA **

WRITE(*,(' SEED ',I10 )) SEED
WRITE(*,(' POTENTIAL ',A )) POTK
WRITE(*,(' DIMENSIONS ',I10 )) DIM
WRITE(*,(' NUMBER OF PARTICLES ',I10 )) NP
WRITE(*,(' NUMBER OF CYCLES ',E10.4 )) DBLE(NSTEP)
WRITE(*,(' NUMBER OF EQUIL. STEPS ',I10 )) IEQUI
WRITE(*,(' OUTPUT FREQUENCY ',I10 )) IPRINT
WRITE(*,(' SAVE FREQUENCY ',I10 )) ISAVE
WRITE(*,(' RATIO UPDATE FREQUENCY ',I10 )) IRATIO
WRITE(*,(' CONFIGURATION FILE NAME ',A )) CNFILE
WRITE(*,(' TEMPERATURE ',E10.4 )) TEMP
WRITE(*,(' DENSITY ',E10.4 )) DENS
WRITE(*,(' PARTICLE MASS ',E10.4 )) FTM
WRITE(*,(' # TIME SLICES ',I10 )) FTNO
WRITE(*,(' TIME STEP ',E10.4 )) LS

C ** CONVERT INPUT DATA TO PROGRAM UNITS **

SIGMA = ( DBLE ( NP ) / DENS ) ** ( 1.0 / DIM )
DRMAX = ALPHA * SIGMA

IF (POTK .EQ. 'LJ' .AND. NMIX .EQ. 1) THEN
  DENSLJ = DENS * SIG(1,1) ** DBLE( DIM )
  RCUT = SIGMA/2.d0/SIG(1,1)
ENDIF

MAXBIN = INT ( SIGMA/DELTA )
MAXBIN2 = INT ( MAXBIN/2.0 )
HIST11 = 0.
HIST12 = 0.
HIST22 = 0.

IF (DIM .EQ. 3) THEN
  CONST11 = 4.d0 * PI * DENS * CO(1)/ 3.d0
  CONST12 = 4.d0 * PI * DENS * CO(2)/ 3.d0
  CONST21 = 4.d0 * PI * DENS * CO(1)/ 3.d0
  CONST22 = 4.d0 * PI * DENS * CO(2)/ 3.d0
ENDIF

IF (DIM .EQ. 2) THEN
  CONST11 = PI * DENS * CO(1)
  CONST12 = PI * DENS * CO(2)
  CONST21 = PI * DENS * CO(1)
  CONST22 = PI * DENS * CO(2)
ENDIF

IF (DIM .EQ. 1) THEN
  CONST11 = 2.d0 * DENS * CO(1)
  CONST12 = 2.d0 * DENS * CO(2)
  CONST21 = 2.d0 * DENS * CO(1)
  CONST22 = 2.d0 * DENS * CO(2)
ENDIF

C ** INITIALIZE CONFIGURATION **

DO I = 1, NP
  NEXT(I) = I
  LEXT(I) = I
  DO J = 1, FTNO
    NEXT(J,I) = I
  ENDDO
ENDDO

C ** ZERO ACCUMULATORS **

ACV = 0.0
ACVSQ = 0.0
FLV = 0.0
ACW = 0.0
ACWSQ = 0.0
FLW = 0.0
ACKE = 0.0
ACKESQ = 0.0
FLKE = 0.0
ACWINDSQ = 0.0
ACM = 0.0
ACATMA = 0.0
ACATMA = 0.0
ACATMAS = 0.0

C ** READ INITIAL CONFIGURATION **
1122 IF ( INIT .EQ. 0 ) THEN
  PRINT*, "PATHS INITIALIZATION ....."
  OPEN (UNIT=9, STATUS='UNKNOWN')
  CALL INITCN ( CNFILE ) ! random configuration
  CALL READCN ( CNFILE ) ! random configuration
  NS = NSTEP
ELSEIF ( INIT .EQ. 1) THEN
  PRINT*, "RESTART FROM CONFIG ....."
  OPEN (UNIT=9, STATUS='UNKNOWN')
  CALL READCN ( CNFILE )
  NS = NSTEP
ELSEIF ( INIT .EQ. 2) THEN
  PRINT*, "RESTART FULL ....."
  OPEN (UNIT=9, ACCESS='APPEND')
  CALL READCN ( CNFILE )
  ACATMA = 0.0
  ACATMA = 0.0
  ACATMAS = 0.0
  READ(20,*)(HIST11(I),I=1,MAXBIN),
  & (HIST12(I),I=1,MAXBIN),
  & (HIST22(I),I=1,MAXBIN),
  & ACV,ACVSQ,ACW,ACWSQ,ACKE,ACKESQ,ACWINDSQ,ACM,NS
  NS=NS+NSTEP
ELSE
  WRITE (*,*) 'INIT different from 0,1,2'
  STOP
ENDIF

```



```

      acatmab = acatmab - 1.0
      if (ip.ne.kp) then
        acatmas = acatmas - 1.0
        call switch(next(ip),next(kp))
        call switch(mext(cp,ip),mext(cp,kp))
      endif
1235      continue

c      build the permutations cycles in lext

      call permcyc()

      IF (STEP.GT.IEQUI) THEN
        ACM = ACM + 1.0

C      ** CALCULATE INSTANTANEOUS VALUES **

        IF (POTK .EQ. 'LJ') THEN
          VN = ( V + VLRC )
        ELSE
          VN = V
          WN = W
        ENDIF
        CALL CWIND( ISP(IP), WIND )

C      ** ACCUMULATE AVERAGES **

        ACV = ACV + VN
        ACVSQ = ACVSQ + VN*VN
        ACW = ACW + WN
        ACWSQ = ACWSQ + WN*WN
        ACKE = ACKE + KE
        ACKESQ = ACKESQ + KE*KE
        DO J=1,NMIX
          DO I=1,DIM
            ACWINDSQ(J,I) = ACWINDSQ(J,I) + WIND(J,I)**2.
          ENDDO
        ENDDO
      ENDIF

      IF (IFBRIDGE) GOTO 97

C      *****
C      ** ENDS BRIDGE&SWAP MOVE **
C      *****

77      CONTINUE

C      *****
C      ** DISPLACEMENT MOVE **
C      *****

C      ** PREVIOUS IP **

      DO I=1,NP
        J=NEXT(I)
        PREV(J)=I
      ENDDO

      NIP = NEXT(IP)
      PIP = PREV(IP)

      DO 90 C1 = 1, FTNO

        DO IDIM = 1, DIM
          RXIOLD(IDIM) = RX(IDIM,C1,IP)
        ENDDO

C      ** CALCULATE THE ENERGY OF I IN THE OLD CONFIGURATION **

        CALL PENERGY ( POTK, RXIOLD, IP, C1,
          :           VOLD, WOLD )

        CALL KENERGY ( RXIOLD, IP, C1,
          :           KEOLD )

C      ** INSTANTANEOUS VALUE OF THE ACTION **

        ACTOLD = KEOLD + VOLD

C      ** MOVE I AND PICKUP THE CENTRAL IMAGE **

        DO IDIM = 1, DIM
          RXINEM(IDIM) = RXIOLD(IDIM) +
          :           ( 2.0 * RANF ( DUMMY ) - 1.0 )*DRMAX
c          RXINEM(IDIM) = RXINEM(IDIM) -
c          :           DNINT ( RXINEM(IDIM)/SIGMA )*SIGMA
          ENDDO

C      ** CALCULATE THE ENERGY OF I IN THE NEW CONFIGURATION **

        CALL PENERGY ( POTK, RXINEM, IP, C1,
          :           VNEW, WNEW )

        CALL KENERGY ( RXINEM, IP, C1,
          :           KENEM )

C      ** INSTANTANEOUS VALUE OF THE ACTION **

        ACTNEW = KENEM + VNEW

        DELTV = VNEW - VOLD
        DELTW = WNEW - WOLD
        DELTKE = KENEM - KEOLD

C      ** CHECK FOR ACCEPTANCE **

        DELTACT = ACTNEW - ACTOLD
        DELTACTB = LS*DELTACT

        IF ( EXP ( - DELTACTB ) .GT. RANF ( DUMMY ) ) THEN

V      = V + DELTV
W      = W + DELTW
KE      = KE + DELTKE
ACATMA = ACATMA + 1.0
DO IDIM = 1, DIM
  RX(IDIM,C1,IP) = RXINEM(IDIM)
C      IMAGINARY TIME PERIODIC BOUNDARY CONDITIONS
  IF (C1.EQ.1) THEN
    RX(IDIM,FTNO+1,PIP)=RX(IDIM,C1,IP)
  ENDIF
  IF (C1.EQ.FTNO) THEN
    RX(IDIM,0,NIP)=RX(IDIM,C1,IP)
  ENDIF
ENDDO

C      RESTRICTED PATH INTEGRAL
DO I=1,FTNO
  SGNI=1.
  DO J=1,NMIX
    CALL IPDW(J,RS,I,SGN2)
    SGNI=SGNI+SGN2
  ENDDO
  IF ( SGNI .LT. 0. ) GOTO 1236
ENDDO
GOTO 1237
CONTINUE
V      = V - DELTV
W      = W - DELTW
KE      = KE - DELTKE
ACATMA = ACATMA - 1.0

DO IDIM = 1, DIM
  RX(IDIM,C1,IP) = RXIOLD(IDIM)
  IF (C1.EQ.1) THEN
    RX(IDIM,FTNO+1,PIP)=RX(IDIM,C1,IP)
  ENDIF
  IF (C1.EQ.FTNO) THEN
    RX(IDIM,0,NIP)=RX(IDIM,C1,IP)
  ENDIF
ENDDO
CONTINUE
ENDIF

IF (STEP.GT.IEQUI) THEN
  ACM = ACM + 1.0

C      ** CALCULATE INSTANTANEOUS VALUES **

C      HISTOGRAM FOR g(r)
      IF ( MOD ( C1, ICALCG ) .EQ. 0 ) THEN
        CALL GOFRC(C1, DENS, MAXBIN, HIST11, HIST12, HIST22)
      ENDIF

      IF (POTK .EQ. 'LJ') THEN
        VN = ( V + VLRC )
      ELSE
        VN = V
        WN = W
      ENDIF
      CALL CWIND( ISP(IP),WIND )

C      ** ACCUMULATE AVERAGES **

        ACV = ACV + VN
        ACVSQ = ACVSQ + VN*VN
        ACW = ACW + WN
        ACWSQ = ACWSQ + WN*WN
        ACKE = ACKE + KE
        ACKESQ = ACKESQ + KE*KE
        DO J=1,NMIX
          DO I=1,DIM
            ACWINDSQ(J,I) = ACWINDSQ(J,I) + WIND(J,I)**2.
          ENDDO
        ENDDO
      ENDIF

C      *****
C      ** END DISPLACEMENT MOVE **
C      *****

90      CONTINUE

C      *****
C      ** ENDS LOOP OVER TIME SLICES **
C      *****

97      CONTINUE

C      ** WRITE OUT THE INSTANTANEOUS VALUES ON FORT.9 **

      SF = 0.0
      SFI = 0.0
      DO I=1, NMIX
        DO IDIM = 1, DIM
          SFI(I) = SFI(I) + WIND(I,IDIM)*WIND(I,IDIM)
          SF(I) = SF(I) + ACWINDSQ(I,IDIM)
        ENDDO
      ENDDO
      DO I=1, NMIX
        SFI(I) = SFI(I)*FTM(I)/(DIM*BETA*NPM(I))
        SF(I) = SF(I)*FTM(I)/(DIM*BETA*NPM(I)*ACM)
      ENDDO
      IF (ISP(IP).EQ.1)THEN
        WSP(1)=1
        WSP(2)=0
      ELSE
        WSP(1)=0
        WSP(2)=1
      ENDIF

      CALL FLUSH(9)

```

```

      IF ( MOD ( STEP, IPRINT ) .EQ. 0 ) THEN
        WRITE(9,*) STEP,DIM*NP/2/LS-KE/FTNO, V/FTNO, W/FTNO,
          SFI,WSP
C      WRITE(9,*) STEP, DIM*NP/2/LS-KE/FTNO, V/FTNO, W/FTNO
      ENDIF

C      ** CREATE POSITION DISTRIBUTION **

C      CALL DISTR(30_8,STEP*FTNO*NP/30)

C      ** PERFORM PERIODIC OPERATIONS **

      IF ( MOD ( STEP, IRATIO ) .EQ. 0 ) THEN

C      ** ADJUST MAXIMUM DISPLACEMENT **

        RATIO = ACATMA / DBLE ( FTNO * IRATIO )
        RATIOB = ACATMAB / DBLE ( IRATIO )
        RATIOS = ACATMAS / DBLE ( IRATIO )

        IF ( RATIO .GT. 0.5 ) THEN

C          DRMAX = DRMAX * 1.05

        ELSE

C          DRMAX = DRMAX * 0.95

        ENDIF

        ACATMA = 0.0
        ACATMAB = 0.0
        ACATMAS = 0.0

      ENDIF

      IF ( MOD ( STEP, IPRINT ) .EQ. 0 ) THEN

C      ** WRITE OUT RUNTIME INFORMATION **

        WRITE(*,*) '          step          acm dr/sig ard
:      arb      ars'
        WRITE(*, '(2(2XI13),(2Xf6.4),4(2Xf8.6))') STEP, INT(ACM),
:              ALPHA, RATIO, RATIOB, RATIOS
        WRITE(*,*) '          ke          v          w
:      sf'
        WRITE(*, '(4(2XE13.7))') KE/FTNO, V/FTNO, W/FTNO, SFI
        WRITE(*,*) '          <ke>          <v>          <w>
:      <sf>'
        WRITE(*, '(4(2XE13.7))') ACKE/ACM/FTNO, ACV/ACM/FTNO,
:              ACW/ACM/FTNO, SF

      ENDIF

      IF ( MOD ( STEP, ISAVE ) .EQ. 0 ) THEN

C      ** WRITE OUT THE CONFIGURATION AT INTERVALS **

        CALL WRITCN ( CNFILE )
        REWIND(20)
        CALL FLUSH(20)
        WRITE(20,*) (HIST11(I), I=1, MAXBIN),
&      (HIST12(I), I=1, MAXBIN),
&      (HIST22(I), I=1, MAXBIN),
&      ACV, ACVSQ, ACW, ACWSQ, ACKE, ACKESQ, ACWINDSQ, ACM, STEP

C      ** WRITE OUT THE g(r) **

      IF ( STEP .GT. IEQUI ) THEN
        TSTEP = DBLE((STEP - IEQUI)*INT(FTNO/ICALOG))
        REWIND(11)
        REWIND(12)
        REWIND(13)
        REWIND(14)
        DO BIN = 1, MAXBIN2
          RLOWER = DBLE ( BIN - 1 ) * DELR
          RUPPER = RLOWER + DELR
          NIDEAL11 = CONST11*( RUPPER**DIM - RLOWER**DIM )
          NIDEAL12 = CONST12*( RUPPER**DIM - RLOWER**DIM )
          NIDEAL22 = CONST22*( RUPPER**DIM - RLOWER**DIM )
          GR11 = DBLE(HIST11(BIN))/TSTEP/DBLE(NPM(1))/NIDEAL11
          GR12 = DBLE(HIST12(BIN))/TSTEP/DBLE(NPM(1))/NIDEAL12
          GR22 = DBLE(HIST22(BIN))/TSTEP/DBLE(NPM(2))/NIDEAL22
          CALL FLUSH(11)
          CALL FLUSH(12)
          CALL FLUSH(13)
          CALL FLUSH(14)
          WRITE (11,*) DBLE(BIN-1)*DELR, GR11
          WRITE (12,*) DBLE(BIN-1)*DELR, GR12
          WRITE (13,*) DBLE(BIN-1)*DELR, GR22
          WRITE (14,*) DBLE(BIN-1)*DELR,
:              CO(1)**2*GR11+2.40*CO(1)*CO(2)*GR12+CO(2)**2*GR22
        ENDDO
      ENDIF

      ENDIF

100  CONTINUE

C      *****
C      ** ENDS THE LOOP OVER CYCLES **
C      *****

      WRITE(*, '(//'' END OF MARKOV CHAIN          ''//)')

C      ** CHECKS FINAL VALUE OF THE POTENTIAL ENERGY IS CONSISTENT **

      CALL SUMUP (POTK, OVRLAP, KEEND, VEND, WEND)

      IF ( ABS(VEND - V) .GT. 1.0d-03 ) THEN

        WRITE(*, '(//'' PROBLEM WITH V ENERGY !!!''')

        WRITE(*, '(// VEND = '', E20.6)') VEND
        WRITE(*, '(// V = '', E20.6)') V

      ENDIF

      IF ( ABS(KEEND - KE) .GT. 1.0d-03 ) THEN

        WRITE(*, '(// PROBLEM WITH KE ENERGY !!!''')
        WRITE(*, '(// KEEND = '', E20.6)') KEEND
        WRITE(*, '(// KE = '', E20.6)') KE

      ENDIF

C      ** WRITE OUT THE FINAL CONFIGURATION FROM THE RUN **

      CALL WRITCN ( CNFILE )

C      ** CALCULATE AND WRITE OUT RUNNING AVERAGES **

      AVV = ACV / ACM
      ACVSQ = ( ACVSQ / ACM ) - AVV ** 2
      AVKE = ACKE / ACM
      ACKESQ = ( ACKESQ / ACM ) - AVKE ** 2

C      ** CALCULATE FLUCTUATIONS **

      IF ( ACVSQ .GT. 0.0 ) FLV = SQRT ( ACVSQ/ACM )/FTNO
      IF ( ACKESQ .GT. 0.0 ) FLKE = SQRT ( ACKESQ/ACM )/FTNO

      WRITE(*, '(// AVERAGES ''/ ' ')
      WRITE(*, '(// <V/N> = '', E12.6)') AVV/FTNO
      WRITE(*, '(// <KE/N> = '', E12.6)') AVKE/FTNO

      WRITE(*, '(// FLUCTUATIONS ''/ ' ')

      WRITE(*, '(// FLUCTUATION IN <V/N> = '', E12.6)') FLV
      WRITE(*, '(// FLUCTUATION IN <KE/N> = '', E12.6)') FLKE
      WRITE(*, '(// END OF SIMULATION ''/ ' ')

      CLOSE (UNIT = 9 )
      CLOSE (UNIT = 11)
      CLOSE (UNIT = 12)
      CLOSE (UNIT = 13)
      CLOSE (UNIT = 14)

C      *****
C      *****
C      ** THE END **
C      *****
C      *****
      STOP
      END

      subroutine permcyc()
      implicit none
!      given a particle i text(i) returns the cycle it belongs to
      INCLUDE 'mc-mix.par'

      integer*8 i,j,k,ii(mnp),jj,kk,ll

      kk=1
      do i=1,np
        ll=1
        ii(ll)=i
        jj=next(ii(ll))
        do j=1,ll
          if(jj.eq.ii(j))goto 2
        enddo
        ll=ll+1
        ii(ll)=jj
        goto 1
      2      do k=1,ll
        text(ii(k))=kk
      enddo
      kk=kk+1
    enddo

      return
      end

!      subroutine switch(i,j)
      switch i and j
      implicit none
      integer*8 i,j,k
      k=i
      i=j
      j=k
      return
      end

      subroutine acc_p(p,ip,kp,j)
      implicit none
!      complete acceptance probability
      INCLUDE 'mc-mix.par'

      integer*8 ip,kp,j,ldim
      real*8 p, rho
      real*8 rxink,rxnik
      real*8 rxini,rxknk
      integer*8 mcm

      p=exp(-ls*p) ! contribution from the pair potential
      if (ip.eq.kp) return

      mcm = j+mbm-floor((j+mbm-1)/ftn0)*ftn0
      rho=0.d0
      do idim=1,dim

```

```

if(j+mbm.le.ftn0)then
  rxink=rx(idim,j,ip)-rx(idim,mcm,kp)
  rxnik=rx(idim,j,kp)-rx(idim,mcm,ip)
  rxini=rx(idim,j,ip)-rx(idim,mcm,ip)
  rxknk=rx(idim,j,kp)-rx(idim,mcm,kp)
  rxink=rxink-dmint(rxink/sigma)*sigma
  rxnik=rxnik-dmint(rxnik/sigma)*sigma
  rxini=rxini-dmint(rxini/sigma)*sigma
  rxknk=rxknk-dmint(rxknk/sigma)*sigma
else
  rxink=rx(idim,j,ip)-rx(idim,mcm,next(kp))
  rxnik=rx(idim,j,kp)-rx(idim,mcm,next(ip))
  rxini=rx(idim,j,ip)-rx(idim,mcm,next(ip))
  rxknk=rx(idim,j,kp)-rx(idim,mcm,next(kp))
  rxink=rxink-dmint(rxink/sigma)*sigma
  rxnik=rxnik-dmint(rxnik/sigma)*sigma
  rxini=rxini-dmint(rxini/sigma)*sigma
  rxknk=rxknk-dmint(rxknk/sigma)*sigma
endif
rho=rho+rxink**2+rxnik**2+rxini**2+rxknk**2
enddo
rho=ftm(isp(ip))*rho/(2.*mbm*ls)
p=p*exp(-rho)
return
end

subroutine update(j,rxp,ip,nip)
implicit none
! updates a portion of the current path x using the proposed path xp
INCLUDE 'mc-mix.par'

integer*8 ip,nip,kp,nkp,j,l,k,idim
integer*8 mcm
real*8 rxp(mdim,0:n)

mcm = j+mbm-floor((j+mbm-.1)/ftn0)*ftn0

l=0
if(j+mbm.le.ftn0)then
  do k=j+1,mcm-1
    l=l+1
    do idim=1,dim
      rx(idim,k,ip)=rxp(idim,l)
    enddo
  enddo
else
  do k=j+1,ftn0
    l=l+1
    do idim=1,dim
      rx(idim,k,ip)=rxp(idim,l)
    enddo
  enddo
  do k=1,mcm-1
    l=l+1
    do idim=1,dim
      rx(idim,k,nip)=rxp(idim,l)
    enddo
  enddo
endif
do idim=1,dim
  rx(idim,ftn0+1,ip)=rx(idim,1,nip)
  rx(idim,0,nip)=rx(idim,ftn0,ip)
enddo

return
end

subroutine swap(j,ip,nip,kp,nkp)
implicit none
! updates a portion of the current path x using the proposed path xp
INCLUDE 'mc-mix.par'

integer*8 ip,nip,kp,nkp,j,k,idim
integer*8 mcm
real*8 rr

mcm = j+mbm-floor((j+mbm-.1)/ftn0)*ftn0

if(j+mbm.le.ftn0)then
  do k=mcm,ftn0
    do idim=1,dim
      rr=rx(idim,k,ip)
      rx(idim,k,ip)=rx(idim,k,kp)
      rx(idim,k,kp)=rr
    enddo
  enddo
  do idim=1,dim
    rx(idim,ftn0+1,ip)=rx(idim,1,nkp)
    rx(idim,ftn0+1,kp)=rx(idim,1,nip)
    rx(idim,0,nip)=rx(idim,ftn0,kp)
    rx(idim,0,nkp)=rx(idim,ftn0,ip)
  enddo
endif
return
end

subroutine bridge(x0,x1,std,xnew,out)
implicit none
! sample n gaussians with std from xnew(0)=x0 to xnew(ftn0)=x1
INCLUDE 'mc-mix.par'

integer*8 l1,l2,l3,j,out,idim
real*8 std,d,s,xi
real*8 x0(mdim),x1(mdim),xnew(mdim,0:n)

out=0
l3=mbm
do idim=1,dim
  xnew(idim,0)=x0(idim)
  xnew(idim,l3)=x0(idim)+((x1(idim)-x0(idim))-
    : dnint((x1(idim)-x0(idim))/sigma)*sigma)
  enddo
  do j=1,mbm-1
    l1=j-1
    l2=j
    s=std*(dble(l3-l2)/dble(l3-l1))**0.5d0
    do idim=1,dim
      d=xnew(idim,l3)-xnew(idim,l1)
      d=d-dnint(d/sigma)*sigma
      xnew(idim,j)=xnew(idim,l1)+d/dble(l3-l1)+xi(s)
      if (xnew(idim,j).gt.sigma/2.or.
c      : xnew(idim,j).lt.-sigma/2) then
c      out=1
c      return
c      endif
c      xnew(idim,j)=xnew(idim,j)-dnint(xnew(idim,j)/sigma)*sigma
    enddo
  enddo
  return
end

function xi(std)
! sample a gaussian with standard deviation std (box-muller method)
implicit none
real*8 xi,std,pi,ranf
data pi/3.14159265358979323846264338328d0/
xi=cos(pi*ranf(0.d0))*std*sqrt(-2.d0*log(tiny(pi)+ranf(0.d0)))
return
end

SUBROUTINE DISTR(NN,NORM)
IMPLICIT NONE
C WRITES ON FORT.10 THE X-POSITION DISTRIBUTION
INCLUDE 'mc-mix.par'

REAL*8 DS,DIST(0:1000)
INTEGER*8 NN,NORM,I,J,K
SAVE DIST

DS=SIGMA/NN

DO I=0,NN
  DO J=1,FTN0
    DO K=1,NP
      IF(-SIGMA/2+(I-.5)*DS.LT.RX(1,J,K).AND.
: RX(1,J,K).LT.-SIGMA/2+(I+.5)*DS) THEN
        DIST(I)=DIST(I)+1.DO
      ENDIF
    ENDDO
  ENDDO
  WRITE(10,*) -SIGMA/2+I*DS,DIST(I)/NORM
ENDDO

CLOSE(UNIT=10)

RETURN
END

FUNCTION FACT ( N )
IMPLICIT NONE
C FACTORIAL FUNCTION
INTEGER*8 FACT,N,P,I
P=1
DO I=1,N
  P=P*I
ENDDO
FACT=P
END

SUBROUTINE SUMUP (POTK, OVRLAP, KE, V, W)
IMPLICIT NONE
C *****
C ** CALCULATES THE TOTAL ENERGY **
C **
C ** USAGE: **
C **
C ** THE SUBROUTINE RETURNS THE TOTAL ENERGY AT THE **
C ** BEGINNING AND END OF THE RUN. **
C *****
INCLUDE 'mc-mix.par'

REAL*8 V, KE, VV, KK
LOGICAL OVRLAP
CHARACTER POTK*(*)

REAL*8 RXII, RXIJ
REAL*8 VIJ, WIJ, RIJSQ, W, WW

INTEGER*8 TAU, I, J, IDIM

C POTENTIAL ACTION

VV = 0.0
WW = 0.0

C ** LOOP OVER ALL THE PAIRS IN THE LIQUID **

DO TAU = 1, FTN0
  DO 100 I = 1, NP - 1
    DO 99 J = I + 1, NP
      RIJSQ = 0.d0
      DO IDIM = 1, DIM
        RXIJ = RX(IDIM,TAU,I) - RX(IDIM,TAU,J)
C ** MINIMUM IMAGE THE PAIR SEPARATIONS **
        RXIJ = RXIJ -

```

```

:          DNINT ( RXIJ/SIGMA )*SIGMA
      RIJSQ = RIJSQ + RXIJ * RXIJ
      ENDDO

      CALL POT ( I, J, RIJSQ, VIJ, WIJ, POTK)
      VV = VV + VIJ
      WW = WW + WIJ
99      CONTINUE
100     CONTINUE
      V=VV
      W=WW
      ENDDO

C      KINETIC ACTION

      KK = 0.0

      DO TAU = 1, FTWO
      DO I = 1, NP
      DO IDIM = 1, DIM
      RXII = RX(IDIM,TAU,I)-RX(IDIM,TAU+1,I)
      RXII = RXII
:          DNINT ( RXII/SIGMA )*SIGMA
      KK=KK+FTM(ISP(I))*(RXII**2.)/(2.DO*LS**2.)
      ENDDO
      ENDDO
      KE=KK
      ENDDO

      RETURN
      END

      SUBROUTINE PENERGY ( POTK, RXI, I, TAU,
:      V, W )
      IMPLICIT NONE
C      *****
C      ** RETURNS THE POTENTIAL ENERGY OF ATOM I WITH ALL OTHER ATOMS. **
C      **
C      ** PRINCIPAL VARIABLES:
C      **
C      ** INTEGER I          THE ATOM OF INTEREST
C      ** INTEGER NP        THE NUMBER OF ATOMS
C      ** INTEGER TAU       THE TIMESTEP
C      ** REAL*8 RX, RY, RZ THE ATOM POSITIONS
C      ** REAL*8 RXI,RVI,RZI THE COORDINATES OF ATOM I
C      ** REAL*8 V          THE POTENTIAL ENERGY OF ATOM I
C      ** REAL*8 W          THE VIRIAL OF ATOM I
C      **
C      ** USAGE:
C      **
C      ** THIS SUBROUTINE IS USED TO CALCULATE THE CHANGE OF ENERGY
C      ** DURING A TRIAL MOVE OF ATOM I. IT IS CALLED BEFORE AND
C      ** AFTER THE RANDOM DISPLACEMENT OF I.
C      *****
      INCLUDE 'mc-mix.par'

      REAL*8 RXI(MDIM), V, W
      INTEGER*8 I, J, TAU, IDIM
      CHARACTER POTK*(*)

      REAL*8 RXIJ, RIJSQ, VIJ, WIJ

      V = 0.0
      W = 0.0

C      ** LOOP OVER ALL MOLECULES EXCEPT I **

      DO 100 J = 1, NP

      IF ( I .NE. J ) THEN
      RIJSQ = 0.0
      DO IDIM = 1, DIM
      RXIJ = RXI(IDIM) - RX(IDIM,TAU,J)

      RXIJ = RXIJ -
:          DNINT ( RXIJ/SIGMA )*SIGMA
      RIJSQ = RIJSQ + RXIJ * RXIJ
      ENDDO

      CALL POT ( I, J, RIJSQ, VIJ, WIJ, POTK)
      V = V + VIJ
      W = W + WIJ
      ENDIF

100     CONTINUE

      RETURN
      END

      SUBROUTINE KENERGY ( RXI, I, TAU, KE )
      IMPLICIT NONE
C      *****
C      ** RETURNS THE KINETIC ENERGY OF ATOM I WITH ALL OTHER ATOMS. **
C      **
C      ** PRINCIPAL VARIABLES:
C      **
C      ** INTEGER I          THE ATOM OF INTEREST
C      ** INTEGER NP        THE NUMBER OF ATOMS
C      ** INTEGER TAU       THE TIMESTEP
C      ** REAL*8 RX, RY, RZ THE ATOM POSITIONS
C      ** REAL*8 RXI,RVI,RZI THE COORDINATES OF ATOM I
C      ** REAL*8 KE         THE KINETIC ENERGY OF ATOM I
C      **
C      ** USAGE:
C      **
C      ** THIS SUBROUTINE IS USED TO CALCULATE THE CHANGE OF ENERGY
C      ** DURING A TRIAL MOVE OF ATOM I. IT IS CALLED BEFORE AND
C      ** AFTER THE RANDOM DISPLACEMENT OF I.
C      *****
      INCLUDE 'mc-mix.par'

      REAL*8 RXI(MDIM), KE
      REAL*8 RXIP, RXIS, RXII
      INTEGER*8 I, TAU, IDIM

      KE = 0.0
      DO IDIM = 1, DIM
      RXIP = RX(IDIM,TAU-1,I)
      RXIS = RX(IDIM,TAU+1,I)
      RXII = RXI(IDIM) - RXIP
      RXII = RXII - DNINT ( RXII/SIGMA )*SIGMA
      KE=KE+0.5*FTM(ISP(I))*(RXII/LS)**2.
      RXII = RXI(IDIM) - RXIS
      RXII = RXII - DNINT ( RXII/SIGMA )*SIGMA
      KE=KE+0.5*FTM(ISP(I))*(RXII/LS)**2.
      ENDDO

```



```

10      DO IDIM = 1, DIM
          RAI(IDIM) = SIGMA*(RANF(0.d0)-.5)
        ENDDO
        DO I1 = 1, NP
            RIJSQ = 0.d0
            DO IDIM = 1, DIM
                RXIJ = RXI(IDIM)-RXII(IDIM,I1)
                RXIJ = RXIJ -
:          DWINT ( RXIJ/SIGMA ) * SIGMA
            RIJSQ = RIJSQ + RXIJ * RXIJ
        ENDDO
        IF (RIJSQ.LE.EPSA(ISP(I),ISP(I1))**2) THEN
            GOTO 10
        ENDIF
        ENDDO
        DO IDIM = 1, DIM
            RXII(IDIM,I) = RXI(IDIM)
        ENDDO
        DO J = 1, FTM*1
            WRITE ( CNUNIT, MYFMT ) I, (RXI(IDIM),IDIM=1,DIM), ISP(I)
            DO IDIM = 1, DIM
                RX(IDIM,J,I) = RXI(IDIM)
            ENDDO
        ENDDO
        WRITE ( CNUNIT,13 ) ''
        WRITE ( CNUNIT,13 ) ''
13      FORMAT(A2)

        CLOSE ( UNIT = CNUNIT )

        RETURN
    END

    SUBROUTINE POT ( I, J, RIJSQ, VIJ, WIJ, POTK)
    IMPLICIT NONE
C *****
C ** SUBROUTINE FOR THE PAIR POTENTIAL **
C *****
    INCLUDE 'mc-mix.par'

    INTEGER*8 I, J, K
    CHARACTER POTK*(*)
    REAL*8 RIJ, RIJSQ, RMIN, RMSQ, RCSQ, SR2, SR6, VIJ, WIJ
    REAL*8 COU3, QHS3, F
    PARAMETER ( COU3 = 4.76015472795910701328763470057d0 )
    PARAMETER ( QHS3 = 24.d0 )

C      PI = ACOS(-1.d0)

    VIJ = 0.d0
    WIJ = 0.d0
    IF (DIM.GT. 3) THEN
        WRITE(*,*) "POTENITAIL CANNOT HAVE DIM > 3 !!!"
        STOP
    ENDIF
    IF (POTK.EQ. 'FREE') RETURN

    IF (POTK.EQ. 'LJ') THEN
        CCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCC
C      W. L. McMillan, Phys. Rev. A 138, 442 (1965) C
        CCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCC
        RMIN = TINY(PI)
        RMIN = 0.d0
        RIJSQ = RIJSQ / (SIG(I,J) * SIG(I,J))
        RMSQ = RMIN*RMIN
        RCSQ = RCUT*RCUT
        IF (RIJSQ.LT. RMSQ) THEN
            RIJSQ = RMSQ
            SR2 = 1.d0 / RIJSQ
            SR6 = SR2 * SR2 * SR2
            VIJ = SR6 * ( SR6 - 1.d0 )
            WIJ = SR6 * ( SR6 - 5.d-1 )
            VIJ = 4.d0 * EPS * VIJ
            WIJ = 48.d0 * EPS * WIJ / 3.d0
        ELSEIF (RIJSQ.GT. RMSQ .AND. RIJSQ.LT. RCSQ) THEN
            SR2 = 1.d0 / RIJSQ
            SR6 = SR2 * SR2 * SR2
            VIJ = SR6 * ( SR6 - 1.d0 )
            WIJ = SR6 * ( SR6 - 5.d-1 )
            VIJ = 4.d0 * EPS * VIJ
            WIJ = 48.d0 * EPS * WIJ / 3.d0
        ENDIF
    ELSEIF (POTK.EQ. 'BUMP') THEN
        RIJ = SQRT( RIJSQ )
        IF (RIJ.LE. SIG(I,J)) THEN
            VIJ = EPSR
        ENDIF
    ELSEIF (POTK.EQ. 'PSW') THEN
        RIJ = SQRT( RIJSQ )
        IF (RIJ.LE. SIG(I,J)) THEN
            VIJ = EPSR
        ELSEIF (RIJ.LE. SIG(I,J)+RCUT .AND. RIJ.GT. SIG(I,J)) THEN
            VIJ = -EPSA(I,J)
        ENDIF
    ELSEIF (POTK.EQ. 'COULOMBHS') THEN
        F=0.
        DO K=1,NMIX
            F = F+SIG(1,K)*NPM(K)/(NPM(K)-1)
        ENDDO
        RIJ = SQRT( RIJSQ )
        IF (RIJ.GT. EPSA(ISP(I),ISP(J))) THEN
            IF (DIM.EQ. 3) THEN
                VIJ = -F*COU3*RCUT/SIGMA/2.
                VIJ = VIJ + RCUT*RIJ
                VIJ = SIG(ISP(I),ISP(J))*RCUT/RIJ
            ELSEIF (DIM.EQ. 2) THEN
                VIJ = -F*(6.-PI*LOG(4.))-4.*LOG(SIGMA/RCUT))/4.
                VIJ = VIJ - LOG(RIJ/RCUT)
                VIJ = -SIG(ISP(I),ISP(J))*LOG(RIJ/RCUT)
            ELSEIF (DIM.EQ. 1) THEN
                VIJ = F*SIGMA/RCUT/4.
                VIJ = VIJ - RIJ/RCUT
                VIJ = -SIG(ISP(I),ISP(J))*RIJ/RCUT
            ENDIF
        ELSE
            RIJ = EPSA(ISP(I),ISP(J))
            IF (DIM.EQ. 3) THEN
                VIJ = -F*COU3*RCUT/SIGMA/2.
                VIJ = VIJ + RCUT*RIJ
                VIJ = SIG(ISP(I),ISP(J))*RCUT/RIJ
            ELSEIF (DIM.EQ. 2) THEN
                VIJ = -F*(6.-PI*LOG(4.))-4.*LOG(SIGMA/RCUT))/4.
                VIJ = VIJ - LOG(RIJ/RCUT)
                VIJ = -SIG(ISP(I),ISP(J))*LOG(RIJ/RCUT)
            ELSEIF (DIM.EQ. 1) THEN
                VIJ = F*SIGMA/RCUT/4.
                VIJ = VIJ - RIJ/RCUT
                VIJ = -SIG(ISP(I),ISP(J))*RIJ/RCUT
            ENDIF
        ENDIF
    ELSEIF (POTK.EQ. 'QHS') THEN
        F=0.
        DO K=1,NMIX
            F = F+SIG(1,K)*NPM(K)/(NPM(K)-1)
        ENDDO
        IF (DIM.EQ. 3) THEN
            IF (RIJSQ.LE. SIG(I,J)*SIG(I,J)) THEN
                VIJ = 10.**10.
            ELSE
                VIJ = -F*QHS3*(SIGMA-SIG(I,J))/SIGMA**3/2./FTM(I)
                VIJ = VIJ + (2.*RIJSQ+SIG(I,J)**2.)/
:              (RIJSQ-SIG(I,J)**2.)*2./2./FTM(I)
:              VIJ = -2.*RIJSQ*(RIJSQ+2.*SIG(I,J)**2.)/
:              (RIJSQ-SIG(I,J)**2.)*3./FTM(I)
            ENDIF
        ELSE
            WRITE(*,*) 'QHS in dimension different from 3'
            STOP
        ENDIF
    ELSEIF (POTK.EQ. 'HARMONIC') THEN
        VIJ = 0.5*RIJSQ/RCUT**2.
    ELSE
        WRITE(*,*) 'NO PAIR POTENTIAL AVAILABLE!'
        STOP
    ENDIF

    RETURN
    END

    SUBROUTINE GOFR ( C, DENS, MAXBIN, HIST11, HIST12, HIST22 )
    IMPLICIT NONE
C *****
C ** CALCULATES THE RADIAL DISTRIBUTION FUNCTION HISTOGRAM. **
C *****
    INCLUDE 'mc-mix.par'

    REAL*8 RXIJ, RIJSQ, RIJ, DENS
    INTEGER*8 C, IDIM, I, J, BIN, MAXBIN
    INTEGER*8 HIST11(MAXBIN), HIST12(MAXBIN), HIST22(MAXBIN)

    DO I = 1, NP - 1
        DO J = I + 1, NP
            RIJSQ = 0.
            DO IDIM = 1, DIM
                RXIJ = RX(IDIM,C,I) - RX(IDIM,C,J)
                RXIJ = RXIJ - ANINT ( RXIJ/SIGMA ) * SIGMA
            RIJSQ = RIJSQ + RXIJ * RXIJ
            ENDDO
C      CALCULATE HISTOGRAM FOR g(r)
            RIJ = SQRT ( RIJSQ )
            BIN = INT ( RIJ/DELR ) + 1

            IF (BIN.LE. MAXBIN) THEN
                IF (ISP(I).EQ. 1 .AND. ISP(J).EQ. 1) THEN
                    HIST11(BIN) = HIST11(BIN) + 2
                ELSEIF (ISP(I).EQ. 2 .AND. ISP(J).EQ. 2) THEN
                    HIST22(BIN) = HIST22(BIN) + 2
                ELSEIF (ISP(I).NE. ISP(J)) THEN
                    HIST12(BIN) = HIST12(BIN) + 1
                ENDIF
            ENDIF
        ENDDO
    ENDDO

    RETURN

```



```

INTEGER*8    DIM, NMIX, ISP(MNP)
REAL*8       RX(MDIM,0:N,MNP)
REAL*8       FTM(MMIX), LS, SIGMA
REAL*8       SIG(MMIX,MMIX), EPSA(MMIX,MMIX), EPSR,EPS,RCUT
INTEGER*8     FTNO, NP, NPM(MMIX)
INTEGER*8     NEXT(MNP), LEXT(MNP), MEXT(N,MNP), MBM

! path
COMMON / BLOCK1 / RX, ISP, DIM, NMIX
! pair-potential parameters
COMMON / BLOCK2 / SIG, EPSR, EPSA, EPS, RCUT
! mass, timestep, box edge, # timeslices, # particles
COMMON / BLOCK3 / FTM, LS, SIGMA, FTNO, NP, NPM
! permutations
COMMON / BLOCK4 / NEXT, LEXT, MEXT, MBM

*****
*** data-mix.in *****
*****

0 spatial dimensions (1,2,3,...<= MDIM) mixture dimension (1,2,3,...<= MMIX)
3 2
1 seed of the random sequence                                RAND
3
2 if bose                                                    (T/F)
T
3 number of particles NPM(I)                                (NP=NPM(1)+NPM(2)+...<= MNP)
10
10
4 the potential                                              SUBROUTINE POT
COULOMB
5 # time slices = 1/temperature/timestep                    (< N)
20
6 masses FTM(1), FTM(2), ...                                (hbar = kb = 1)
1.d0
1836.15d0
7 # of cycles                                                (nstep)
5000000000000000000
8 # of steps between output lines                            (iprint)
100
9 # of steps between configuration saves                      (isave)

100
10 # of steps for equilibration                               (iequi)
100
11 # of steps for acceptance ratios                           (iratio)
100
12 # of steps for rdf calculation                             (icalcg < #5)
10
13 configuration file name
conf.xyz
14 0 initialization, 1 restart, 2 restart rdf
0
15 density                                                    THERMODYNAMICS
-16d0
16 temperature                                                THERMODYNAMICS
-005d0
17 maximum displacement/box edge
-01d0
18 maximum # of bridge timeslices                             (> 1; <= #5)
20
19 potential cutoff distance (LJ)                             SUBROUTINE POT
1.d0
20 sig(i,j)                                                    (LJ 2.566) SUBROUTINE POT
1.d0
-1.d0
-1.d0
1.d0
21 epsr                                                        (LJ INIT) SUBROUTINE POT
1.d0
22 epsa(i,j)                                                  SUBROUTINE POT
0.d0
-5d0
-5d0
1.d0
23 eps                                                        (LJ 10.22) SUBROUTINE POT
1.d1
24 if only displace                                           (T/F)
F
25 if only bridge                                             (T/F)
F

```

-
- [1] D. M. Ceperley, Path integrals in the theory of condensed Helium, *Rev. Mod. Phys.* **67**, 279 (1995).
- [2] D. M. Ceperley, Path integral Monte Carlo methods for fermions, in *Monte Carlo and Molecular Dynamics of Condensed Matter Systems*, edited by K. Binder and G. Ciccotti (Editrice Compositori, Bologna, Italy, 1996).
- [3] M. H. Kalos and P. A. Whitlock, *Monte Carlo Methods* (John Wiley & Sons Inc., New York, 1986).
- [4] R. Fantoni, Coherent State Path Integral Monte Carlo, *Eur. Phys. J. D* (2025), under review.
- [5] D. M. Ceperley, Fermion Nodes, *J. Stat. Phys.* **63**, 1237 (1991).